

Patterns and anti-patterns of efficient software

Edited by

Oscar Palm¹

¹ Chalmers University of Technology, Sweden nilsosc@chalmers.se

1 Introduction

An anti-pattern, as described by Brown et al. [1], is a method of recognizing recurring problems in software engineering, and providing a general solution and a common vocabulary for identification and solving these recurring problems. This chapter intends to present and discuss anti-patterns within the field of performance engineering, as well as the common solutions, or patterns, for problems within the field. For the readers' benefit, patterns and anti-patterns have been grouped into categories, and in some cases similar patterns have been combined.

To present the patterns in a structured way, a modified version of the template presented by Brown et al. in their book *AntiPatterns* [1], has been used.

The patterns and anti-patterns have been divided into several categories, representing during which type of work the pattern is useful, or the anti-pattern is commonly found. Since more patterns were found than can be discussed in a single chapter, a list of all found patterns, and which literature more can be read about them in, can be found in Tables 1 and 2.

The purpose of this chapter is to understand common performance issues in software engineering, and how these issues can be solved through planning and good design. The patterns will also help understand common issues within the field, helping maintainers find problems, and understand ways of solving them.

2 Communication

2.1 Network Communication

Following are patterns relating to different applications communicating, both over networks, but also within the same computer. Most of these patterns discuss how to efficiently use relevant I/O devices.

Since sending and receiving of data can be time consuming when communicating between computers, the communication itself can become a performance issue. Many patterns and anti-patterns discuss this communication between computers and we can roughly divide them into two groups, where one group discusses the data in each request, and the other group discusses how the requests themselves can be handled more efficiently.

2.1.1 Data Handling in Network Communication

For the first group of patterns, an example is the Empty Semi Truck anti-pattern, which discusses how what logically might be several distinct requests sometimes can be combined into one request, to limit the idling time of a client.



Except where otherwise noted, content of this report is licensed under a Creative Commons BY 3.0 Unported license

Special Topics in Software Engineering, *Dagstuhl Reports*, Vol. 1, Issue 2025, pp. 1–11

Editor: Oscar Palm



Dagstuhl Reports

Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl Publishing, Germany

Anti-Pattern Name: Empty Semi Truck [9, 2]

Solution Name: Batching of requests

Background

This antipattern occurs when a client sends more requests than necessary, slowing down execution because of idling time waiting for the unnecessary amount of responses [2].

Empty Semi Trucks are caused by one of two main issues; a poorly written interface, or an inefficient use of the bandwidth [9].

Solution

For the first case, a poorly written interface, allowing for coupling of several request into one helps decrease the amounts of requests needed [2]. Depending on what currently is forcing the client to send multiple requests, what needs to be rewritten can change, but examples can be allowing for clients to fetch entire objects, instead of separate requests for different parts of it, or allowing the fetching of several objects at once.

If the problem instead is using the bandwidth inefficiently, we can instead use batching to combine several requests into one [2]. This helps lower the overhead of sending and receiving several messages.

2.1.2 Request Handling in Network Communication

The other group of patterns and anti-patterns related to network communication discuss how to handle requests; is it possible to perform other tasks while waiting for responses? Can we send multiple requests at once? Many of the patterns and anti-patterns discuss using asynchronous computing to better utilize computing resources.

Commonly, software is written synchronously, meaning the previous instruction needs to complete before continuing execution; with network communication however, the time of some instructions may take a very significant amount of time. Several Patterns and Anti-patterns discuss ways of utilising the computer to avoid idly waiting for these requests to complete.

Anti-Pattern Name: Synchronous I/O [6]

Solution Name: Asynchronous messaging

Background

The program is blocked by a synchronous I/O call. No useful work is completed while the I/O operation completes. Often this occurs while requesting data from web services or fetching data from discs.

Solution

Convert the synchronous I/O operation to an asynchronous one, if possible, execution of other operations can continue while the resulting I/O action is completing. While this does not always solve the problem; some times data from the I/O operation is needed to continue, often times the result from such operations are not used, think storing data; in which case synchronosity is consuming time unnecessarily.

Pattern Name: Asynchronous Request-Reply [5]

Also Known As: Polling [7]

Issue To Fix: Synchronous web calls leading to unnecessarily long load times.

Background

In modern web applications, it's common to use data or services from external APIs, such as to show advertisement, handle authentication, or to access certain data. Instead of the server not returning until these requests have been completed and the result ready, http polling can be used.

With polling, the server responds immediately with status 202 accepted. When the client later needs to access the result, it polls the server, which instead can either return code 302, value found, and redirect to where the result can be found, or code 200, the result is not ready yet, after which the client can poll again later.

While this in itself might not mitigate the problem with long wait times, if the client is structured smartly, data can be requested before it is needed, freeing up the client to perform other tasks while the request is completing.

2.2 Internal communication

These patterns relate to issues regarding communication between different modules, classes, functions, or other divisors within a software system.

Since the communication is most likely happening between systems with smaller request latency, the main focus is not the requests any more, but rather what data is sent. The shorter messaging time means performance issues more likely arise from handling the requests rather than waiting for the requests to complete.

While some of the patterns and anti-patterns, such as Tower of Babel, discuss minimizing the cost of communication in different ways, others instead recognize the limits of vertical scalability.

Patterns, such as the Choreography pattern, instead discuss ways of offloading some of the requests to other instances of the same service.

Both of these types of improving efficiency of the software is necessary. When only focusing on horizontal scaling of systems, inefficient use of system resources might still be a problem. This might make larger requests bottleneck the system. Likewise if only focusing on vertical scaling, the hardware used limits the efficiency of a system heavily.

Anti-Pattern Name: Tower of Babel [9, 2]

Solution Name: Minimize parsing and translation

Background

When sending data between components, it is common to translate said data into common formats, such as JSON, XML, or CSV. It is however common for both the receiving and sending component to use its own internal representation of the data, sometimes even the same internal representation in both components.

This conversion between different data formats can be costly, especially when not only two components are communicating, but rather a chain of several components.

Solution

Using the fast path performance pattern [9] we notice which paths are in need of streamlining. For these paths, we rewrite the code to minimize the amount of translation between data representations.

When data is sent within the system, make sure all parts use the same internal representation, instead of translation between different internal representations and / or export formats such as XML [9].

Pattern Name: Choreography [5]

Issue To Fix: When a system is under heavy use, the need for scalability becomes more prominent. With direct communication between different services within the system, it can be hard to scale the system efficiently [5].
Instead routing requests through an orchestrator requires a very complex architecture and often relies on custom third party software, such as kubernetes.

Background

Instead of using direct communication or orchestrators, a queue of tasks can be used. With a queue based system, services can poll the queue to see if the task is something they can process or not.

This pattern is similar to the Competing Customers pattern [5], and Worker Pool pattern[12], with the distinction of each worker being able to represent a different service. This means that while a worker in the worker pool is identical to all other workers, in the choreography pattern, each service can be either different instances of the same service, or its own distinct service.

3 Data handling

This group of patterns are related to access of, and storage of, data. Whether it is related to data stored in files on the disk, or in a database, these patterns help you manage access to it.

Similar to network communication, accessing of data is time consuming, especially when said data is not stored locally.

Pattern Name: Cache-aside [5]

Issue To Fix: Accessing data from a database takes time, storing the data locally is not an option either since data in the database might get updated over time.

Background

Caching is used to improve the performance of an application by storing commonly used information in a server temporarily instead of always requesting the data from an external party [11].

It can be hard to manage a cache, since data over time becomes outdated, or stale, without the client knowing. This issue of knowing when to update the cache is a challenge in development of distributed systems [5].

Instead of loading data from the database, or other similar services, this pattern suggest implementing a cahche, where each data request first checks if the data exist in the cache before requesting data externally [5].

When data is being fetched, it is also added to the cache in case it is needed later on [5].

4 Parallel Execution

Parallelism is a tool often used to increase performance of applications[4], but if used incorrectly, its benefits might not come into effect and in the worst case, a performance decrease can even be seen.

The More is less anti pattern is a great example of how parallelism can hurt the performance of an application because of the extra overhead added when parallelizing a task.

Even when parallelism is implemented in a way, helping the performance of an application, it is easy to create threads with uneven workloads, leading to large parts of the theoretically parallelisable application being executed sequentially. The Competing Customers pattern discusses how this problem can disappear, by modifying the architecture of our system.

Anti-Pattern Name: More is less [2]
Solution Name: Less forking.

Background

A process creating sub-processes to perform parallel execution in some cases ends up spending more time handling threads than performing actual work.

Solution

The solution to execution time slowing because of "over-parallelizing" is to limit the amount of allowed processes. Since the limits of when thrashing occurs is dependent on both hardware and the parallelized software [3], we test the specific combination of hardware and software, to quantify which threshold for amount of processes where thrashing occurs [2].

Based on the results from this test, we can then decide a limit for amount of processes, or in case of the architecture not meeting performance goals, modify the architecture until performance goals are met.

Pattern Name: Competing Customers [5]
Also Known As: Worker Pool pattern [12]
Issue To Fix: When a module in a system sees heavy use, and some requests take a long time to process, it might be frustrating for users when a seemingly simple request takes much longer than usual just because their task got assigned the "wrong" thread.

Background

By using a message queue, several instances of the service can work on the tasks simultaneously. This allows for easier horizontal scaling of the service, since instances can be created or removed without interrupting the rest of the system.

Usage of the message queue also improves reliability since a failure in a consumer, or worker, does not mean all pending messages will be lost, unlike with direct communication between producer and consumer [5].

Usage of this pattern also removes the issue of uneven workload distribution, since all processes share the same backlog of tasks.

5 Orchestration

For hosting of services, it is common to use orchestrators to help divide and manage their resources. When the amount of users of a service grows, it is also common to have several instances of a service, either to share the processing load, or to minimize access time by distributing instances over different servers spread across the globe. Chaotic session management is an anti-pattern that might appear when using several instances of an application without accounting for the separate memory of different instances.

When orchestrating systems, comprised of several different services, or instances of the same service, there is also a need to allow clients to connect to all of these, it can be hard to remember which service does what, or to know which instance of a service to connect to. The Gateway Routing Pattern helps balance loads and connect clients to relevant servers without the need to understand the internal architecture.

Anti-Pattern Name: Chaotic session management [10]
Solution Name: Dispatching with session affinity

Background

When managing sessions over several different instances of a system, the session of a user in one instance is not immediately transferred to other instances, and we do not always want to send a user's session information to all instances of the system.

To the user however, if the next request they send are routed to another instance of the service, it will look as if their session has been terminated. There needs to exist a better way to handle sessions in a system with multiple instances.

Solution

One easy solution to this problem is to always dispatch requests from the same user to the same instance; if the user always communicates with the same server, the distributedness of the architecture is hidden [10].

Some issues still persist with this solution however. If an instance fails, all users assigned to that instance will lose their session. Since one of the major benefits of having several instances of a system is that (hopefully) the reliability is higher, because other instances of the service can replace failing ones; this is a major flaw in the solution. However, other solutions which mitigate this issue comes with performance penalties [10], meaning this is still one of the better solutions to the problem.

Pattern Name: Gateway Routing [5]
Issue To Fix: With a distributed system, it can be tricky remembering which API does what, and the name of the services.
It can also be tricky knowing how many instances of a service exists and which one to connect to.

Background

By having a gateway service, that routes each request to the relevant service, and instance

of that service, clients need only connect to a single API, hiding implementation logic, and exact details of which instance the user is connected to [5].

Usage of this pattern allows for easy horizontal scaling of services, as well for increased reliability, since after a failure in one of the instances, requests can be routed through the gateway to other instances, without the need for clients to repeat the request.

It however introduces a single point of failure, if an issue happens in the gateway, access to all services are lost [5]. Similarly since the gateway is a service in itself, it can also be overloaded; if it is not fast enough, it can itself become a performance issue [5].

6 Execution Ordering

The order different tasks are executed in can affect the performance of an application, if the tasks are executed in a bad order, a parallelized program can be forced to execute partly serialized.

While tasks such as garbage collection and backupper are important, they take time. A Website that does not consider its execution order might schedule a system backup while a system tries accessing a web page, severely worsening the load time of the page, possibly losing customers.

The Priority Queue pattern discusses how tasks can be given different priorities to help manage the order they will be executed.

Pattern Name: Priority Queue [5]

Issue To Fix: Tasks handled by a system is not all of the same level of importance, having critical tasks delayed, because of relatively unimportant tasks in the queue, can cost a lot of money for companies, or in worst case lead to death, in case of healthcare software.

Background

By assigning each task a priority, we can make sure that critical tasks are handled before not as critical tasks. There's two main ways of creating a priority queue, either there's one list of all queued tasks, where each task is given a priority and the queue sorts its item after priority. The other option is to create one queue per priority level, making sure the highest-level queue is empty before working on tasks from lower priorities [5].

While a single queue works fine for most cases, it can become slow to add tasks when the queue grows large, as it needs a way to sort itself.

Using multiple queues has many benefits; for one, the existence of several queues means instances of a service can be assigned different priority levels, making sure that while the higher priority queue still have priority, the lower priority tasks still resolves [5].

7 Miscellaneous

Some patterns and anti-patterns doesn't fit into a grouping of problem areas as well as others. Since these are still important to consider while developing and refactoring they are still included in the chapter.

While most patterns and anti-patterns discussed focus on the execution time of application, other factors also affect the performance of an application. The Blob anti-pattern is one of these, discussing how poorly written code can affect the memory usage of applications [1].

Anti-Pattern Name: Blob [1, 2]
Also Known As: The God Class [2]
Solution Name:

Background

The Blob is a class which does more than it is supposed to, instead of a system containing classes with data, and functionality related to the data, systems with a blob generally contain data classes, with limited functionality, and a larger "Blob" which performs most of the operations these data classes should include.

From a performance perspective, Blobs are concerning since the loading of large classes is memory heavy, meaning loading all data for the Blob, every time a simple operation is performed, can slow execution time [1].

Solution

Since the performance degradation is due to excessive loading of data into memory, the solution is to lower the amount of data loaded into memory. To do this, functionality regarding data from other classes should be moved to those classes [2].

It can also help to remove associations to other classes, moving those associations to other classes instead [1].

Name	Type	Category	Source
Asynchronous Request-Reply	Pattern	Communication	[5]
Backends for Frontends	Pattern	Communication	[5]
Bulkhead	Pattern	Communication	[5]
Circuit Breaker	Pattern	Communication	[5]
Queue-Based Load Leveling	Pattern	Communication	[5]
Choreography	Pattern	Communication	[5]
Sidecar	Pattern	Communication	[5]
Cache-aside	Pattern	Data Handling	[5]
Claim Check	Pattern	Data Handling	[5]
Event Sourcing	Pattern	Data Handling	[5]
Index Table	Pattern	Data Handling	[5]
Materialized View	Pattern	Data Handling	[5]
Competing Customers	Pattern	Parallel Execution	[5]
Gateway Aggregation	Pattern	Orchestration	[5]
Gateway Offloading	Pattern	Orchestration	[5]
Gateway Routing	Pattern	Orchestration	[5]
Health Endpoint Monitoring	Pattern	Orchestration	[5]
Scheduler Agent Supervisor	Pattern	Orchestration	[5]
Sharding	Pattern	Orchestration	[5]
Throttling	Pattern	Orchestration	[5]
Priority Queue	Pattern	Execution Ordering	[5]
Publisher / Subscriber	Pattern	Execution Ordering	[5]
Federated Identity	Pattern	Miscellaneous	[5]

■ **Table 1** All found Patterns

Name	Type	Category	Source
Synchronous I/O	Anti-pattern	Communication	[6]
Falling Dominoes	Anti-pattern	Communication	[9]
Empty Semi Truck	Anti-pattern	Communication	[9],[2]
Round-tripping	Anti-pattern	Communication	[10]
Busy Database	Anti-pattern	Communication	[6]
Busy Front End	Anti-pattern	Communication	[6]
Chatty I/O	Anti-pattern	Communication	[6]
Retry Storm	Anti-pattern	Communication	[6]
Too many Remote Calls	Anti-pattern	Communication	[8]
Tower of Babel	Anti-pattern	Communication	[9],[2]
Improper Instantiation	Anti-pattern	Communication	[6]
Excessive Dynamic Allocation	Anti-pattern	Communication	[2]
Aggressive Loading of Entities	Anti-pattern	Communication	[8]
Circuitous Treasure Hunt	Anti-pattern	Communication	[2]
No Caching	Anti-pattern	Data Handling	[6]
Extraneous Fetching	Anti-pattern	Data Handling	[6]
Monolithic Persistence	Anti-pattern	Data Handling	[6]
Concurrent Processing Systems	Anti-pattern	Parallel Execution	[2]
"Pipe and Filter" Architectures	Anti-pattern	Parallel Execution	[2]
One-Lane Bridge	Anti-pattern	Parallel Execution	[2]
More is Less	Anti-pattern	Parallel Execution	[2]
Chaotic Session Management	Anti-pattern	Orchestration	[10]
Noisy Neighbor	Anti-pattern	Orchestration	[6]
Traffic Jam	Anti-pattern	Orchestration	[2]
Bad Workload Management	Anti-pattern	Execution Ordering	[10]
Performance Afterthoughts	Anti-pattern	Miscellaneous	[10]
Blob	Anti-pattern	Miscellaneous	[2],[1]
Extensive Processing	Anti-pattern	Miscellaneous	[2]
The Ramp	Anti-pattern	Miscellaneous	[2]

■ **Table 2** All found Anti-patterns

8 Conclusion

This chapter has discussed various types of patterns as well as how they can be applied in real-world scenarios. Where different anti-patterns can be found and how to deal with them have also been discussed.

While most patterns found are related to development of distributed systems, such as web applications, the same patterns can also be applied to other types of system.

The chapter also provides a common vocabulary to discuss problems and problem solutions within performance engineering, helping developers communicate issues and teams of developers understand the choices made by other people. While patterns do not always provide the best solution for a specific case, the generality and widespread solution description helps create a maintainable code base.

A recurring theme throughout the chapter is the effect of performance bottlenecks. The bottlenecks are created by inefficient communication, inefficient resource use, or executing

tasks without prioritization. Many of these patterns do not focus on improving the code, but rather on balancing the work, and utilizing the computer's resources in a better way.

Many of the anti-patterns discussed instead describe how laziness during planning can lead to problems such as Blobs [1, 2] or Synchronous I/O [6], which in turn creates issues for the performance of an application.

References

- [1] William J. Brown et al. *AntiPatterns: Refactoring Software, Architectures, and Projects in Crisis*. John Wiley & Sons, 1998, p. 336. ISBN: 978-0-471-19713-3.
- [2] Vittorio Cortellessa, Antinisca Di Marco and Catia Trubiani. "An approach for modeling and detecting software performance antipatterns based on first-order logics". In: *Software & Systems Modeling* 13.1 (2014), pp. 391–432. DOI: 10.1007/s10270-012-0246-z.
- [3] Peter J Denning. "Thrashing: Its causes and prevention". In: *Proceedings of the December 9-11, 1968, fall joint computer conference, part I*. 1968, pp. 915–922.
- [4] Louis Mak. "Parallelism always helps". In: *SIAM Journal on Computing* 26.1 (1997), pp. 153–172.
- [5] Microsoft Learn. *Architecture design patterns that support performance efficiency*. Last updated: 2024-10-10; Accessed: 2025-11-20. Oct. 2024. URL: <https://learn.microsoft.com/en-us/azure/well-architected/performance-efficiency/design-patterns>.
- [6] Microsoft Learn. *Performance testing and antipatterns for cloud applications*. Last updated: 2023-12-13. Accessed: 2025-11-23. 2023. URL: <https://learn.microsoft.com/en-us/azure/architecture/antipatterns/>.
- [7] Mohamed-Ali. *Understanding Short Polling Model in Modern Web Architecture*. Accessed: 2025-12-10. 2025. URL: <https://medium.com/@moali314/understanding-short-polling-model-in-modern-web-architecture-3cc65170249c>.
- [8] Trevor Parsons. "A framework for detecting, assessing and visualizing performance antipatterns in component based systems". In: *Companion to the 19th Annual ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages, and Applications*. OOPSLA '04. Vancouver, BC, CANADA: Association for Computing Machinery, 2004, pp. 316–317. ISBN: 1581138334. DOI: 10.1145/1028664.1028781. URL: <https://doi.org/10.1145/1028664.1028781>.
- [9] Connie Smith and Lloyd Williams. "More New Software Antipatterns: Even More Ways to Shoot Yourself in the Foot." In: Dec. 2003, pp. 717–725.
- [10] Bruce A. Tate and Braden R. Flowers. *Bitter Java*. Manning Publications Co., 2002. ISBN: 1-930110-43-X.
- [11] Duane Wessels. *Web Caching: Reducing Network Traffic*. Sebastopol, CA: O'Reilly & Associates, 2001. ISBN: 1-56592-536-X.
- [12] Eduardo Zepeda. *Worker Pool Design Pattern Explanation*. Posted on DEV Community; originally published at coffeebytes.dev on Jun 21, 2024. June 2024. URL: https://dev.to/zeedu_dev/worker-pool-design-pattern-explanation-3kil.